

AI 동화책 생성 서비스 — 프론트엔드 연동 API 명세서

버전: 2.0.0
작성일: 2026-05-13

1. 기본 정보

항목	값
서버 URL	별도 전달 예정
응답 포맷	JSON (기본), SSE, PNG, WAV
CORS	모든 Origin 허용
인증	없음

API 문서 (Swagger) [{서버URL}/docs](#)

- 기본 URL (로컬): <http://127.0.0.1:8000>
- 기본 URL (외부): <https://mystorybook.local.lt>
- Swagger UI: <http://127.0.0.1:8000/docs>
- 응답 포맷: JSON, SSE, PNG, WAV
- CORS: 모든 origin, method, header 허용

2. 엔드포인트 목록

Method	Path	설명
GET	/health	서버 상태 확인
POST	/api/stt/field	음성 파일 → 텍스트 변환 (STT)
POST	/api/runs	동화 생성 작업 시작
GET	/api/runs/{run_id}	작업 상태 및 결과물 URL 조회
GET	/api/runs/{run_id}/events	작업 진행 상황 실시간 스트림 (SSE)
GET	/api/runs/{run_id}/story	생성된 스토리 JSON 조회
GET	/api/runs/{run_id}/images/{filename}	생성 이미지 다운로드
GET	/api/runs/{run_id}/audio/{filename}	생성 오디오 다운로드

3. 엔드포인트 상세

3-1. GET [/health](#)

서버 동작 여부를 확인합니다.

응답 예시

```
{ "status": "ok" }
```

3-2. POST /api/stt/field — 음성 → 텍스트 변환

입력 폼의 각 필드를 음성으로 입력할 때 사용합니다. 녹음 파일을 서버에 업로드하면 Whisper STT로 텍스트를 반환합니다.

Request

Content-Type: multipart/form-data

필드	타입	필수	설명
audio_file	File	O	녹음 파일. webm, mp4, mp3, wav 지원
field_type	string	O	어느 입력란인지 지정. era / place / characters / topic 중 하나
language	string	X	언어 코드. 기본값 ko-KR

응답 예시

```
{
  "stt_text": "조선 시대",
  "parsed_value": "조선 시대",
  "confidence": 0.91
}
```

필드	타입	설명
stt_text	string	Whisper 원문 인식 결과
parsed_value	string	입력란에 넣을 최종 값 (이 값을 사용하세요)
confidence	number	인식 신뢰도. 0.0 ~ 1.0

에러

상태 코드	원인
400	field_type 값이 올바르지 않음 / 빈 음성 파일
500	STT 처리 실패

3-3. POST /api/runs — 동화 생성 시작

사용자 입력을 받아 동화 생성 파이프라인을 시작합니다. 생성은 백그라운드에서 비동기로 진행됩니다.

Request

Content-Type: application/json

```
{
  "era_ko": "조선 시대",
  "place_ko": "한양의 작은 서당",
  "characters_ko": "호기심 많은 아이와 말하는 호랑이",
  "topic_ko": "서로를 믿는 우정"
}
```

필드	타입	필수	설명
era_ko	string	O	시대
place_ko	string	O	장소
characters_ko	string	O	등장인물
topic_ko	string	O	주제

응답 — 201 Created

```
{
  "run_id": "20260429_173423_d118aa"
}
```

run_id는 이후 모든 조회 요청에서 사용합니다. 반드시 저장하세요.

3-4. GET /api/runs/{run_id} — 작업 상태 조회

작업의 현재 상태와 생성된 미디어 URL을 반환합니다. 폴링 또는 SSE와 병행하여 사용합니다.

응답 예시 (생성 완료 후)

```
{
  "run_id": "20260429_173423_d118aa",
  "status": "DONE",
  "stage": "IMAGE",
  "story_title": "말하는 호랑이와 작은 약속",
  "cover": {
    "scene_no": 0,
    "status": "End",
    "scenarios": [
      {
        "index": 0,
        "msg": "",
        "audio_url": ""
      }
    ]
  }
}
```

```

        "image_url": "/api/runs/20260429_173423_d118aa/images/cover.png",
        "delay": 0
    },
    {
        "index": 1,
        "msg": "말하는 호랑이와 작은 약속",
        "audio_url": "/api/runs/20260429_173423_d118aa/audio/cover.wav",
        "image_url": "",
        "delay": 5
    }
]
},
"scenes": [
    {
        "scene_no": 1,
        "status": "End",
        "scenarios": [
            {
                "index": 0,
                "msg": "",
                "audio_url": "",
                "image_url":
"/api/runs/20260429_173423_d118aa/images/scene_01_img_01.png",
                "delay": 4
            },
            {
                "index": 1,
                "msg": "한양의 작은 서당 앞에 커다란 발자국이 남아 있었습니다.",
                "audio_url": "/api/runs/20260429_173423_d118aa/audio/scene_01_0.wav",
                "image_url": "",
                "delay": 8
            },
            {
                "index": 2,
                "msg": "",
                "audio_url": "",
                "image_url":
"/api/runs/20260429_173423_d118aa/images/scene_01_img_02.png",
                "delay": 4
            },
            {
                "index": 3,
                "msg": "누가 이렇게 큰 발자국을 남긴 걸까?",
                "audio_url": "/api/runs/20260429_173423_d118aa/audio/scene_01_1.wav",
                "image_url": "",
                "delay": 3
            }
        ]
    }
],
"error": null
}

```

최상위 응답 필드

필드	타입	설명
run_id	string	작업 ID
status	string	작업 상태. 아래 표 참고
stage	string	현재 처리 단계. 아래 표 참고
story_title	string	동화 제목. LLM 완료 전에는 빈 문자열
cover	object	표지 시나리오 정보. 아래 표 참고
scenes	array	씬 목록. 항상 4개 . LLM 완료 전에는 빈 배열 []
error	string null	실패 시 에러 메시지

status 값

값	설명
QUEUED	작업 생성 직후, 파이프라인 시작 전
RUNNING	파이프라인 실행 중
DONE	전체 생성 완료
FAILED	처리 실패

stage 값

값	설명
LLM	스토리 텍스트 생성 중
PARALLEL	음성(TTS)과 이미지를 병렬 생성 중
IMAGE	전체 완료 또는 파일 복구 상태

cover 필드 (표지)

cover는 scene_no: 0인 ScenarioInfo 객체입니다. 씬과 동일한 구조를 가집니다.

필드	타입	설명
scene_no	number	항상 0
status	string	Pending / Running / End
scenarios	array	2개 (index 0~1)
index	포함 데이터	설명
0	image_url	표지 이미지 (delay: 0)

index	포함 데이터	설명
1	msg, audio_url, delay	제목 텍스트 + 제목 낭독 TTS. delay = 해당 WAV 길이(초)

scenes[] 필드

필드	타입	설명
scene_no	number	씬 번호. 1 ~ 4
status	string	Pending / Running / End
scenarios	array	재생 시퀀스. 항상 4개 (index 0~3)

scenarios[] 필드

필드	타입	설명
index	number	재생 순서. 0 ~ 3
msg	string	자막 텍스트
audio_url	string	오디오 URL
image_url	string	이미지 URL
delay	number	다음 단계로 넘어가기 전 대기 시간(초)

scenarios 인덱스별 역할

index	포함 데이터	설명
0	image_url, delay	첫 번째 이미지 표시 후 delay초 대기
1	msg, audio_url, delay	내레이션 자막 + 내레이션 오디오 재생. delay = 해당 WAV 길이(초)
2	image_url, delay	두 번째 이미지로 전환 후 delay초 대기
3	msg, audio_url, delay	대사 자막 + 대사 오디오 재생. delay = 해당 WAV 길이(초). 대사가 없으면 audio_url은 빈 문자열, delay는 0

delay 의미: 모든 index에서 delay는 해당 스텝 완료 후 다음 스텝으로 넘어가기 전 대기해야 할 초(seconds)입니다.

- index 0, 2 (이미지): $\text{round}(\text{내레이션 WAV 길이} \div \text{이미지 수})$, 최솟값 1초
- index 1 (내레이션 오디오): $\text{round}(\text{내레이션 WAV 길이})$, 최솟값 0초
- index 3 (대사 오디오): $\text{round}(\text{대사 WAV 길이})$, 대사 없으면 0

오디오 구조: 씬당 WAV가 2개로 분리됩니다. scene_NN_0.wav = 내레이션, scene_NN_1.wav = 대사. 대사 (dialogue)가 없는 씬은 _1.wav가 생성되지 않으므로 audio_url이 빈 문자열입니다.

예러

상태 코드	원인
404	존재하지 않는 <code>run_id</code>

3-5. GET /api/runs/{run_id}/events — 실시간 SSE 스트림

Server-Sent Events로 작업 진행 상황을 실시간으로 수신합니다.

응답 헤더: `Content-Type: text/event-stream`

이벤트 종류

이벤트명	설명
<code>update</code>	상태 변경 또는 미디어 생성 완료 알림
<code>keepalive</code>	연결 유지용. 30초마다 전송. data 없음

update 이벤트 데이터 구조

기본 필드 (항상 포함):

필드	타입	설명
<code>run_id</code>	string	작업 ID
<code>status</code>	string	현재 작업 상태
<code>stage</code>	string	현재 처리 단계

상황별 추가 필드:

필드	발생 조건	설명
<code>cover_image</code>	표지 이미지 생성 완료	표지 이미지 파일명
<code>cover_image_url</code>	표지 이미지 생성 완료	표지 이미지 URL
<code>cover_audio</code>	표지 TTS 완료	표지 오디오 파일명
<code>cover_audio_url</code>	표지 TTS 완료	표지 오디오 URL
<code>scene_no</code>	씬 미디어 완료	해당 씬 번호
<code>image</code>	씬 이미지 1개 완료	이미지 파일명 (<code>scene_no</code> 와 함께)
<code>image_url</code>	씬 이미지 1개 완료	이미지 URL (<code>scene_no</code> 와 함께)
<code>audio</code>	씬 내레이션 TTS 완료	내레이션 오디오 파일명 (<code>scene_no</code> 와 함께)
<code>audio_url</code>	씬 내레이션 TTS 완료	내레이션 오디오 URL (<code>scene_no</code> 와 함께)
<code>dialogue_audio</code>	씬 대사 TTS 완료	대사 오디오 파일명 (<code>scene_no</code> 와 함께)
<code>dialogue_audio_url</code>	씬 대사 TTS 완료	대사 오디오 URL (<code>scene_no</code> 와 함께)

필드	발생 조건	설명
error	status === "FAILED"	에러 메시지

이벤트 흐름 예시

```

event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"LLM"}

event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"PARALLEL"}

event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "cover_audio":"cover.wav", "cover_audio_url":"/api/runs/.../audio/cover.wav"}

event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "scene_no":1, "audio":"scene_01_0.wav", "audio_url":"/api/runs/.../audio/scene_01_0.wav"}

event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "scene_no":1, "dialogue_audio":"scene_01_1.wav", "dialogue_audio_url":"/api/runs/.../audio/scene_01_1.wav"}

event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "cover_image":"cover.png", "cover_image_url":"/api/runs/.../images/cover.png"}

event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "scene_no":2, "image":"scene_02_img_01.png", "image_url":"/api/runs/.../images/scene_02_img_01.png"}

event: update
data: {"run_id":"...", "status":"DONE", "stage":"IMAGE"}

event: keepalive
data:

```

브라우저 연결 예시

```

const source = new EventSource("/api/runs/20260429_173423_d118aa/events");

source.addEventListener("update", (e) => {
  const data = JSON.parse(e.data);

```



```
if (data.status === "DONE" || data.status === "FAILED") {
  source.close();
}
});
```

3-6. GET /api/runs/{run_id}/story — 스토리 JSON 조회

LLM이 생성한 원본 스토리 데이터(제목, 씬별 내레이션/대사/이미지 프롬프트 등)를 반환합니다.

항목	값
응답 형식	application/json

에러

상태 코드	원인
404	존재하지 않는 run_id 또는 아직 LLM 미완료

3-7. GET /api/runs/{run_id}/images/{filename} — 이미지

생성된 PNG 이미지를 반환합니다.

항목	값
응답 형식	image/png
캐시	Cache-Control: no-store

파일명 패턴

```
cover.png
scene_01_img_01.png ~ scene_04_img_02.png
```

에러

상태 코드	원인
400	파일명에 .., /, \ 포함
404	아직 생성되지 않은 파일

3-8. GET /api/runs/{run_id}/audio/{filename} — 오디오

생성된 WAV 오디오를 반환합니다.

항목	값
응답 형식	audio/wav
포맷	RIFF PCM, 비압축
캐시	Cache-Control: no-store

파일명 패턴

```
cover.wav           ← 표지 제목 낭독 TTS
scene_01_0.wav ~ scene_04_0.wav ← 씬 내레이션 TTS
scene_01_1.wav ~ scene_04_1.wav ← 씬 대사 TTS (대사가 있는 씬만 생성됨)
```

에러

상태 코드	원인
400	파일명에 .., /, \ 포함
404	아직 생성되지 않은 파일

4. 공통 에러 응답

필드 누락 등 유효성 오류 (422 Unprocessable Entity)

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "era_ko"],
      "msg": "Field required",
      "input": {}
    }
  ]
}
```

리소스 없음 (404 Not Found)

```
{
  "detail": "Run 20260429_173423_d118aa not found"
}
```

5. 클라이언트 구현 흐름

전체 흐름

1. POST /api/runs → run_id 수신
2. GET /api/runs/{id}/events SSE 연결 (실시간 알림)
+ GET /api/runs/{id} 폴링 병행 권장 (2500ms 간격)
3. scenes.length === 4 감지 → LLM 완료. 씬 카드 4개 렌더링
4. 각 씬 scene.status === "End" 감지 → 해당 씬 재생 가능
5. status === "DONE" → 전체 완료

진행률 계산 예시

전체 생성 결과물은 다음과 같이 구성됩니다.

분류	항목	개수
스토리	story_title 1개 + scenes 4개	5
이미지	cover 1개 + 씬 이미지 2개 × 4씬	9
오디오	cover 1개 + 씬 내레이션 4개 + 씬 대사 최대 4개	5 ~ 9
합계		19 ~ 23

```
// 진행률 (%) 계산 예시
const storyDone = (data.story_title ? 1 : 0) + data.scenes.length;

const imageDone = (data.cover?.scenarios[0]?.image_url ? 1 : 0)
+ data.scenes.reduce((n, scene) => {
  const imgs = scene.scenarios.filter(s => s.image_url).length;
  return n + imgs;
}, 0);

const audioDone = (data.cover?.scenarios[1]?.audio_url ? 1 : 0)
+ data.scenes.reduce((n, scene) => {
  // index 1 = 내레이션, index 3 = 대사
  const hasNarr = scene.scenarios.find(s => s.index === 1)?.audio_url ? 1 : 0;
  const hasDial = scene.scenarios.find(s => s.index === 3)?.audio_url ? 1 : 0;
  return n + hasNarr + hasDial;
}, 0);

const total = 19; // 최소 기준 (대사 없는 씬 포함)
const percent = Math.round(((storyDone + imageDone + audioDone) / total) * 100);
```

씬 재생 구현

모든 스텝에서 delay가 0보다 크면 해당 초만큼 대기하고, 0이면 즉시 다음 스텝으로 넘어갑니다.

```
async function playScene(scene) {
  for (const step of scene.scenarios) {
    if (step.image_url) {
```

```

    showImage(step.image_url);          // 이미지 전환
  }
  if (step.audio_url) {
    showSubtitle(step.msg);             // 자막 표시
    playAudio(step.audio_url);          // 오디오 재생 시작 (await 하지 않음)
  } else if (step.msg) {
    showSubtitle(step.msg);             // 자막만 표시 (대사 오디오 없는 경우)
  }
  if (step.delay > 0) {
    await wait(step.delay * 1000);      // delay초 대기 후 다음 스텝
  }
}
}

```

delay 기반 타이밍: 오디오 **onended** 이벤트를 기다리지 않습니다. 서버가 미리 계산한 **delay** 값(WAV 길이 기반)만큼 대기하면 됩니다. index 3의 **delay**가 0이면 대사 오디오가 없는 씬이므로 자막만 표시 후 즉시 다음 씬으로 넘어가세요.

표지 화면

cover 객체의 **scenarios**를 활용합니다.

```

const coverImage = data.cover?.scenarios[0]?.image_url; // 표지 이미지
const coverTitle = data.cover?.scenarios[1]?.msg;       // 동화 제목 텍스트
const coverAudio = data.cover?.scenarios[1]?.audio_url; // 제목 낭독 오디오

```

표지는 씬 재생과 별개로 독립적으로 표시됩니다.

6. 생성 파이프라인 구조 (참고)

서버는 두 머신이 병렬로 동화를 생성합니다.

```

POST /api/runs 요청
├── LLM on 5080 worker (스토리 텍스트 생성)
│   └── 완료 시 scenes 배열 4개 반환
└── PARALLEL (동시 진행)
    ├── 워커(5080): 표지 TTS → 씬 TTS (내레이션/대사 분리) → 씬 2,3 이미지
    └── 마스터(3080): 표지 이미지 → 씬 1,4 이미지

```

이미지 분배표

씬	담당	이미지
cover	마스터(3080)	cover.png

씬	담당	이미지
scene 1	마스터(3080)	scene_01_img_01.png, scene_01_img_02.png
scene 2	워커(5080)	scene_02_img_01.png, scene_02_img_02.png
scene 3	워커(5080)	scene_03_img_01.png, scene_03_img_02.png
scene 4	마스터(3080)	scene_04_img_01.png, scene_04_img_02.png

stage: "PARALLEL" 상태에서 이미지와 오디오가 도착하는 순서는 불규칙합니다. SSE 이벤트나 폴링으로 각 미디어가 완료되는 시점에 맞춰 UI를 업데이트하세요.

7. 구현 시 주의사항

1. **URL은 상대경로:** `image_url`, `audio_url` 등 모든 미디어 URL은 `/api/runs/...` 형태의 상대경로입니다. 요청 시 서버 Origin을 붙여서 사용하세요.
2. **이미지/오디오 캐시 방지:** 서버가 `Cache-Control: no-store`를 반환하지만, 브라우저 캐시 문제가 발생할 경우 URL에 타임스탬프 쿼리를 추가하세요.

```
/api/runs/.../images/cover.png?t=1746959400000
```

3. **SSE 재연결:** `EventSource`는 연결이 끊기면 브라우저가 자동으로 재연결합니다. `onerror` 이벤트로 재연결 상태를 UI에 표시하는 것을 권장합니다.
4. **폴링 병행:** SSE 연결이 불안정한 환경을 고려해 `GET /api/runs/{run_id}` 폴링(2500ms 권장)을 SSE와 병행하면 안정적입니다.
5. **scenes 배열은 LLM 완료 전 빈 배열:** `data.scenes.length === 0`인 동안에는 씬 영역에 로딩 상태를 표시하세요.
6. **미디어 URL은 생성 완료 후에만 유효:** `image_url` 또는 `audio_url`이 빈 문자열인 경우 아직 생성 중입니다. 값이 있을 때만 요청하세요.
7. **대사 오디오는 선택적:** `scene.scenarios[3].audio_url`은 해당 씬에 대사(dialogue)가 있을 때만 채워집니다. 빈 문자열인 경우 자막(msg)만 표시하세요.
8. **cover는 항상 존재:** `data.cover`는 생성 시작 즉시 반환되며 (status: `Pending`), 각 미디어가 완료될수록 `Running` → `End`로 전환됩니다.